

# **BloodLink Blood Bank Management System**

## **Final System Project Documentation Paper**

**Prepared by:** Jamaica Mae N. Malaki

**Course, Yr. & Sec.:** BSIT 3-B

**Instructor:** Jeremiah Dela Cruz

**Objective:** The purpose of this documentation paper is to present a complete written record of the system project developed throughout the semester. It describes the system's design, development, implementation, database structure, testing procedures, and possible future improvements.

# 1. Introduction

## 1.1 Background of the Study

Blood donation and blood supply management are important public health activities. Hospitals, rural health units, and blood service organizations need accurate records of donors, available blood units, requests, and donation schedules. When these records are handled manually, staff may experience delays, duplicate entries, and difficulty finding available blood during urgent situations.

BloodLink is a web-based Blood Bank Management System developed to organize blood donation and blood request activities. The system provides separate functions for donors, medical facilities, and administrators so that blood inventory, donor appointment requests, and facility blood requests can be monitored in one application.

## 1.2 Purpose of the System

The purpose of BloodLink is to provide an organized platform for managing donor registration, donation requests, facility blood requests, inventory records, and administrative reports. The system aims to make blood bank information easier to store, update, retrieve, and monitor.

# 2. Objectives of the System

## 2.1 General Objective

The general objective of the project is to develop a web-based Blood Bank Management System that helps donors, medical facilities, and administrators manage blood donation activities, blood requests, inventory records, and reports in an organized and accessible platform.

## 2.2 Specific Objectives

- To provide role-based registration and login for donors, medical facilities, and administrators.
- To allow donors to submit blood donation appointment requests.
- To allow medical facilities to approve, reject, schedule, and complete donor requests.
- To allow medical facilities to request blood from other facilities.
- To allow facilities to post and update blood inventory by blood type and component.

- To display available blood inventory for users.
- To provide administrative dashboards and reports for monitoring users, requests, donations, inventory, and low-stock records.
- To store important records in a structured database using Laravel migrations and models.

### 3. Scope and Limitations

#### 3.1 Scope of the System

- Role-based registration for donor, facility, and admin users.
- Login and logout using Laravel session handling.
- Donor profile creation with blood type, age, gender, weight, and declaration confirmation.
- Donation request submission and status tracking for donors.
- Facility review, scheduling, rejection, and completion of donor requests.
- Facility-to-facility blood request submission with urgency level and reason.
- Blood inventory posting by facility, blood type, and component.
- Public blood availability viewing.
- Admin dashboard and reports for monitoring users, donors, facilities, inventory, donations, blood requests, and low-stock alerts.

#### 3.2 Limitations of the System

- The system is intended for local deployment and academic demonstration using Laravel and a local database environment.
- It does not include SMS notification, email notification, online payment, or mobile application support.
- It does not perform automatic medical eligibility checking or laboratory screening.
- It does not provide full real-time integration with external hospital systems.
- Inventory accuracy depends on facility staff updating records correctly.
- The current version does not include a complete audit trail for every record change.

### 4. System Description

BloodLink is a Laravel-based web application that manages blood donation and blood request workflows. It uses role-based access so that each user type can access only the functions

intended for that role. The system uses Laravel routes, controllers, Blade views, Eloquent models, and database migrations to organize the application.

| Module                | Main Functions                                                                                                                                                 |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication Module | Handles role selection, registration, login, logout, password hashing, and session storage.                                                                    |
| Donor Module          | Allows donors to register donor information, submit donation requests, view request status, and check blood availability.                                      |
| Facility Module       | Allows facilities to manage donor requests, schedule appointments, complete donations, submit blood requests, handle addressed requests, and update inventory. |
| Admin Module          | Allows administrators to view dashboards, monitor records, check low stock, and access reports.                                                                |
| Inventory Module      | Stores and displays blood units by facility, blood type, and component.                                                                                        |
| Reports Module        | Summarizes donors, facilities, inventory, donations, requests, users, and monthly data for monitoring.                                                         |

## 5. Technologies Used

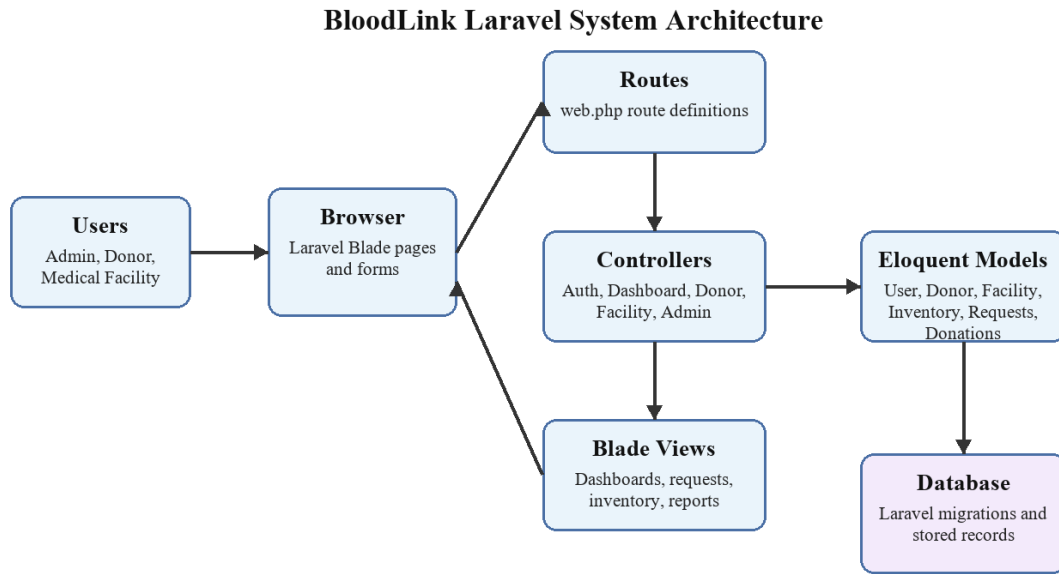
| Technology | Use in the System                                                                                           |
|------------|-------------------------------------------------------------------------------------------------------------|
| PHP        | Main server-side programming language used by Laravel.                                                      |
| Laravel    | Framework used for routing, controllers, models, validation, sessions, migrations, and database operations. |

| Technology                      | Use in the System                                                                                                 |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Blade                           | Templating engine used to create system pages and dashboards.                                                     |
| HTML, CSS, JavaScript           | Used for the user interface, page layout, and frontend behavior.                                                  |
| Vite and NPM                    | Used for frontend asset support and package management.                                                           |
| MySQL / SQLite-ready migrations | Database structure is defined through Laravel migrations and can be used with the configured database connection. |
| Composer                        | Used to manage PHP dependencies.                                                                                  |
| Visual Studio Code              | Used as the development environment.                                                                              |
| Git and GitHub                  | Used for source code management and project repository submission.                                                |

## 6. System Design

### 6.1 System Architecture

The system follows the Model-View-Controller pattern used by Laravel. Users interact with the application through a web browser. Routes receive requests and send them to controllers. Controllers validate input, apply role checks, call Eloquent models, and return Blade views. Models communicate with the database tables where records are stored.



*Figure 1. BloodLink Laravel System Architecture Diagram*

| Layer              | Description                                                                                                                                 |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Presentation Layer | Blade views display the home page, login, registration forms, dashboards, request forms, inventory pages, and reports.                      |
| Controller Layer   | AuthController, DashboardController, DonorController, FacilityController, and AdminController process requests and apply application logic. |
| Model Layer        | User, Donor, MedicalFacility, BloodInventory, DonationRequest, BloodRequest, and Donation represent the main database records.              |
| Database Layer     | Laravel migrations define the tables, fields, keys, and relationships used by the system.                                                   |

## 6.2 Use Case Summary

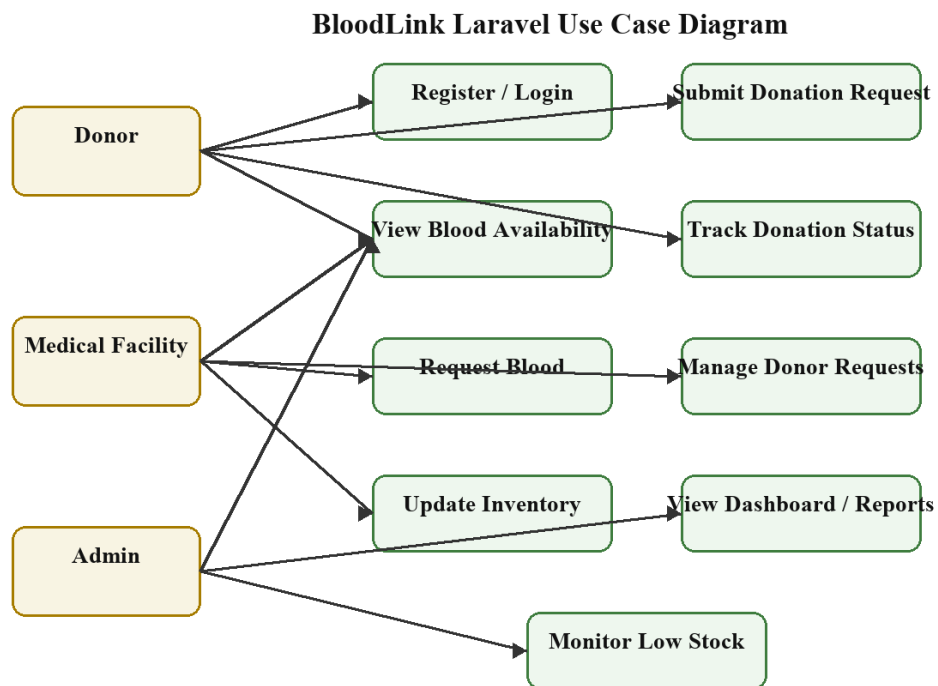


Figure 2. BloodLink Laravel Use Case Diagram

| User Role        | Use Cases                                                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Admin            | View dashboard summaries, monitor users and facilities, view requests, access reports, and check low-stock blood inventory.                            |
| Donor            | Register as donor, log in, submit donation request, view donation request status, and view blood availability.                                         |
| Medical Facility | Register facility profile, request blood, manage addressed facility requests, schedule donor requests, complete donations, and update blood inventory. |

## 6.3 Main System Workflow

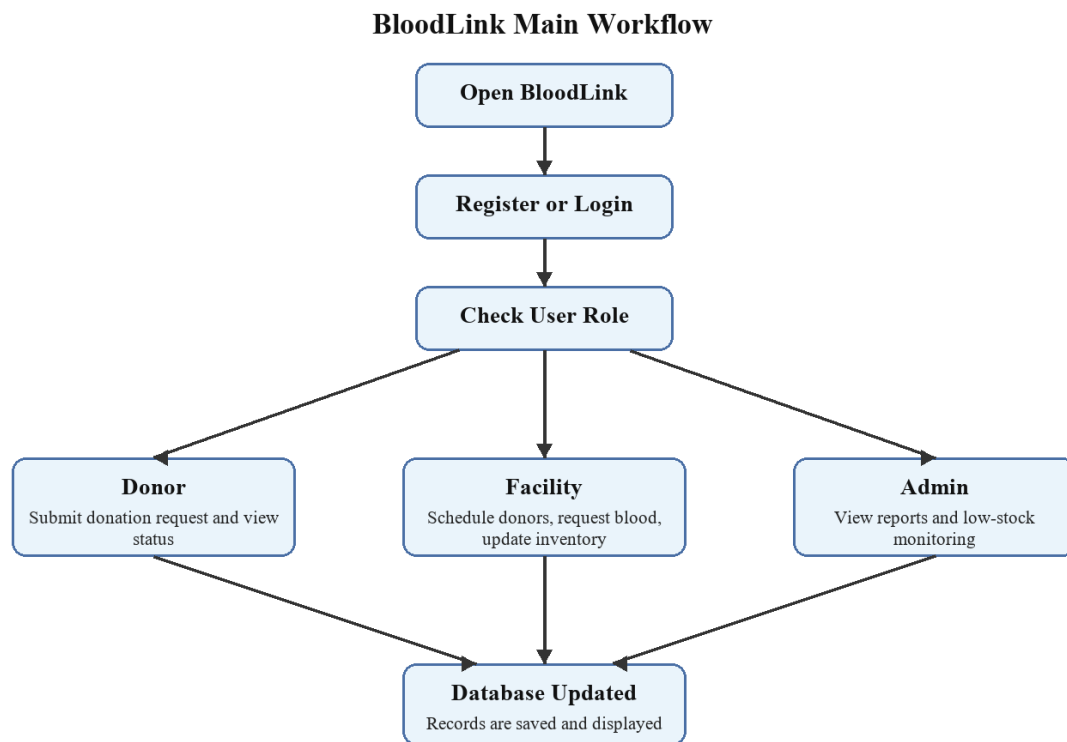


Figure 3. BloodLink Main Workflow / Flowchart

## 7. System Features and Functional Requirements

| Feature                  | Description                                                                                                      |
|--------------------------|------------------------------------------------------------------------------------------------------------------|
| Role-Based Access        | The system separates functions for admin, donor, and medical facility users.                                     |
| Donor Request Management | Donors can submit appointment requests, while facilities can approve, reject, schedule, or complete them.        |
| Blood Request Management | Facilities can request blood from other facilities and update request status as approved, rejected, or released. |
| Inventory Management     | Facilities can add blood units by blood type and component. Available inventory can be viewed by users.          |



| Feature                        | Description                                                                                                                  |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Dashboard and Reports          | The admin dashboard and reports summarize records such as donors, facilities, inventory, donations, requests, and low stock. |
| Validation and Session Control | Laravel validation and session data are used to protect forms and role-specific pages.                                       |

## 8. Database Structure

The database structure is based on Laravel migration files and Eloquent models. The main tables are users, donors, medical\_facilities, blood\_inventories, donation\_requests, blood\_requests, and donations. Laravel's default password\_reset\_tokens and sessions tables are also included for account and session support.

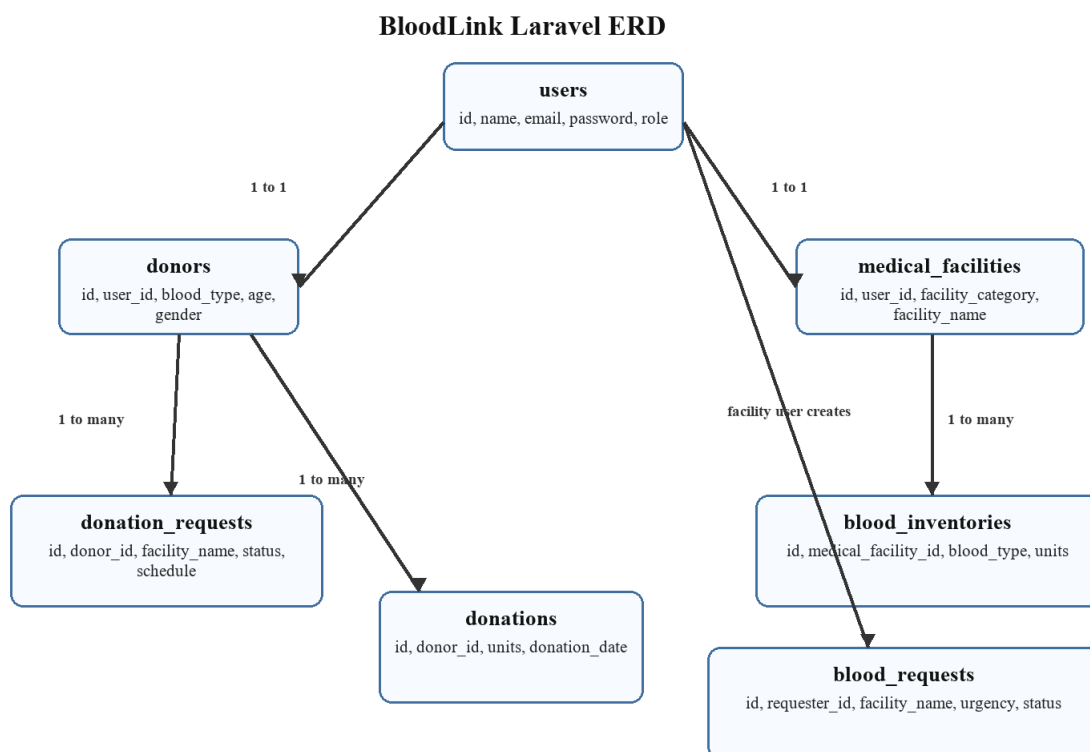


Figure 4. BloodLink Entity Relationship Diagram

## 8.1 List of Tables

| Table Name            | Purpose                                                                  |
|-----------------------|--------------------------------------------------------------------------|
| users                 | Stores account information, login credentials, role, phone, and address. |
| donors                | Stores donor-specific medical and profile information.                   |
| medical_facilities    | Stores registered facility information.                                  |
| blood_inventories     | Stores available blood units by facility, blood type, and component.     |
| donation_requests     | Stores donor appointment requests and their statuses.                    |
| blood_requests        | Stores blood requests submitted by facilities.                           |
| donations             | Stores completed donation records.                                       |
| password_reset_tokens | Stores password reset token data if password reset support is used.      |
| sessions              | Stores Laravel session records when database session storage is enabled. |

## 8.2 Description of Fields

| Table | Important Fields                                                                                           | Description                        |
|-------|------------------------------------------------------------------------------------------------------------|------------------------------------|
| users | id, name, email, email_verified_at, password, role, phone, address, remember_token, created_at, updated_at | User account and role information. |

| Table              | Important Fields                                                                                                                                                              | Description                                |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| donors             | id, user_id, blood_type, age, gender, weight, declaration_confirmed, declaration_confirmed_at, medical_notes, created_at, updated_at                                          | Donor profile linked to a user account.    |
| medical_facilities | id, user_id, facility_category, facility_name, license_number, contact_person, created_at, updated_at                                                                         | Facility profile linked to a user account. |
| blood_inventories  | id, medical_facility_id, facility_name, blood_type, component, units_available, expiry_date, notes, created_at, updated_at                                                    | Blood stock information per facility.      |
| donation_requests  | id, donor_id, facility_category, facility_name, blood_type, component, units, status, scheduled_date, start_time, end_time, donor_note, facility_note, created_at, updated_at | Donation appointment request details.      |
| blood_requests     | id, requester_id, requester_role, facility_category, facility_name, blood_type, component, units, urgency, status, reason, admin_note, created_at, updated_at                 | Facility blood request details.            |
| donations          | id, donor_id, blood_type, component, units, donation_date, facility_name, notes, created_at, updated_at                                                                       | Completed donation records.                |

### 8.3 Relationships Between Tables

- A user may have one donor profile through donors.user\_id.

- A user may have one medical facility profile through `medical_facilities.user_id`.
- A donor may submit many donation requests through `donation_requests.donor_id`.
- A donor may have many completed donation records through `donations.donor_id`.
- A medical facility may own many blood inventory records through `blood_inventories.medical_facility_id`.
- A facility user may create many blood requests through `blood_requests.requester_id`.
- Blood requests and donation requests are addressed to facilities using `facility_category` and `facility_name`.

## 9. Implementation and Testing

### 9.1 Development Process

1. Set up the Laravel project environment and installed required dependencies through Composer and NPM.
2. Configured the local database connection and prepared Laravel migration files.
3. Created database tables for users, donors, medical facilities, blood inventories, donation requests, blood requests, and donations.
4. Built Eloquent models to represent the main data records.
5. Developed authentication pages for role selection, registration, login, and logout.
6. Created role-based dashboards and navigation for admin, donor, and facility users.
7. Implemented donor request submission and donor status viewing.
8. Implemented facility workflows for donor scheduling, blood requests, addressed request updates, and inventory posting.
9. Implemented admin dashboards and reports for monitoring system activity.
10. Tested the system locally using Laravel Artisan serve and sample records.

### 9.2 Testing Procedures Performed

| Test Area    | Procedure                                                                      | Expected Result                                        |
|--------------|--------------------------------------------------------------------------------|--------------------------------------------------------|
| Registration | Registered donor and facility accounts using the role-based registration form. | Accounts are saved with correct role and profile data. |

| Test Area           | Procedure                                                                          | Expected Result                                                                        |
|---------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Login               | Logged in using valid and invalid credentials.                                     | Valid users can access dashboards; invalid credentials are rejected.                   |
| Role Access         | Accessed donor, facility, and admin pages using different roles.                   | Users can access only role-appropriate pages.                                          |
| Donor Request       | Submitted donation request with blood type, component, units, and target facility. | Request is stored with pending status.                                                 |
| Facility Scheduling | Approved donor request with scheduled date, start time, and end time.              | Schedule is saved and invalid time ranges are rejected.                                |
| Donation Completion | Marked an approved donor request as completed.                                     | A completed donation record is created.                                                |
| Blood Request       | Submitted and updated facility blood requests.                                     | Request status changes to approved, rejected, or released according to workflow rules. |
| Inventory           | Added blood units by blood type and component.                                     | Inventory units increase and are displayed in availability records.                    |
| Reports             | Viewed dashboard totals, reports, and low-stock summaries.                         | Counts and lists reflect stored database records.                                      |

### 9.3 Problems Encountered

- Designing a database structure that supports different user roles without duplicating account data.
- Controlling access so donors, facilities, and administrators see only their own allowed functions.
- Validating donor schedules so appointment start time is earlier than end time.
- Managing inventory by both blood type and blood component.
- Keeping donation request statuses and facility blood request statuses consistent.
- Preparing dashboards and reports that summarize records accurately.

### 9.4 Solutions Applied

- A role field was added to the users table to separate admin, donor, and facility accounts.
- Role checks were added in controllers before protected functions are displayed or processed.
- Laravel validation rules were used for required fields, accepted declarations, numeric units, valid dates, and correct time formats.
- Separate tables were created for donors, facilities, inventories, donation requests, blood requests, and donations.
- Eloquent models were used to simplify database operations and relationships.
- Dashboard queries were used to count users, donors, facilities, inventory units, pending requests, recent donations, and low-stock records.

## 10. Conclusion

The BloodLink Blood Bank Management System achieved its goal of providing a web-based platform for organizing blood donation and blood request activities. The system supports role-based registration and login, donor request submission, facility scheduling, facility blood requests, inventory management, availability viewing, and administrative monitoring.

The system benefits donors by giving them a clear way to submit donation requests and monitor their status. It benefits medical facilities by helping them manage donor appointments, blood inventory, and blood request workflows. It benefits administrators by providing dashboards and reports that summarize system activity and stock conditions.

During development, important learning was gained in Laravel routing, controllers, models, migrations, Blade views, validation, sessions, role-based access control, database design, and GitHub project submission. The project also showed the importance of organized records and workflow management in healthcare-related systems.

## **11. Recommendations**

The following improvements are recommended for future development and system enhancement:

- Add email or SMS notifications for donor schedules, approvals, and urgent blood needs.
- Add automatic blood compatibility checking based on blood type rules.
- Add expiry alerts for blood inventory records.
- Add stronger admin approval for facility account verification.
- Add downloadable PDF reports and printable summaries.
- Add search and filtering tools for donors, facilities, requests, and inventory.
- Add audit logs for inventory changes and request status updates.
- Add location or map support to help users find nearby facilities.
- Add backup and restore features for database safety.
- Add laboratory screening and donor eligibility result recording for completed donations.